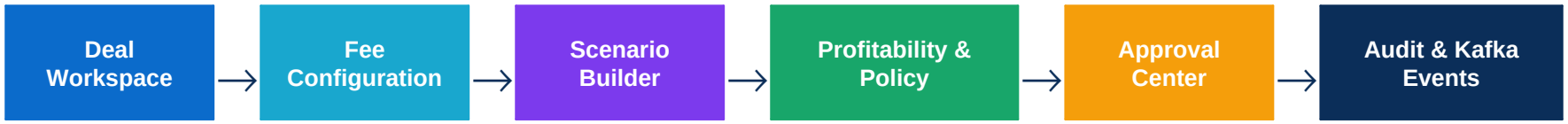
Pricing Studio - Design Rationale

How the front-end business journey maps to clear back-end decision capabilities



Core principle

The front end follows the user's journey; the back end follows the business decision boundaries.

The design starts with business decisions - not with a predetermined number of microservices.

1. Front End vs. Back End: Why the Boundaries Exist

FRONT-END JOURNEY	BUSINESS QUESTION	BACK-END CAPABILITY
Deal Workspace	What are we pricing?	Deal Service
Fee Configuration	How should we price it?	Pricing Engine
Scenario Builder	What alternatives do we have?	Scenario / Pricing Engine
Profitability & Policy	Does it make economic sense, and is it allowed?	Profitability + Policy Engines
Approval Center	Who has authority to approve exceptions?	Workflow / Approval Engine
Audit & Kafka Events	What happened, and who needs to know?	Audit + Kafka Event Backbone

UI captures intent
React guides Sales through the deal lifecycle. It should not contain authoritative pricing or policy logic.

Back end owns decisions
Pricing, profitability, policy and workflow are separated because they answer different business questions.

Logical before physical
These are capability boundaries first. They can begin in a modular Spring Boot deployment and be extracted when justified.

2. Front-End Rationale: Follow the Business Journey

1	**Deal Workspace**	Create one canonical deal context: client, products, AUM, volumes, currency and relationship attributes.
2	**Fee Configuration**	Capture commercial intent: fee basis, tiers, minimums, discounts and bundles. The UI does not own calculation logic.
3	**Scenario Builder**	Compare Standard, Proposed and AI-recommended alternatives before requesting approval.
4	**Profitability & Policy**	Bring economic results and governance results together for the user, while keeping the engines separate.
5	**Approval Center**	Show exactly which functions must act - Finance, Product, Risk - and the current workflow state.
6	**Audit & Events**	Make the deal explainable: what was calculated, what was approved, and which events were emitted.

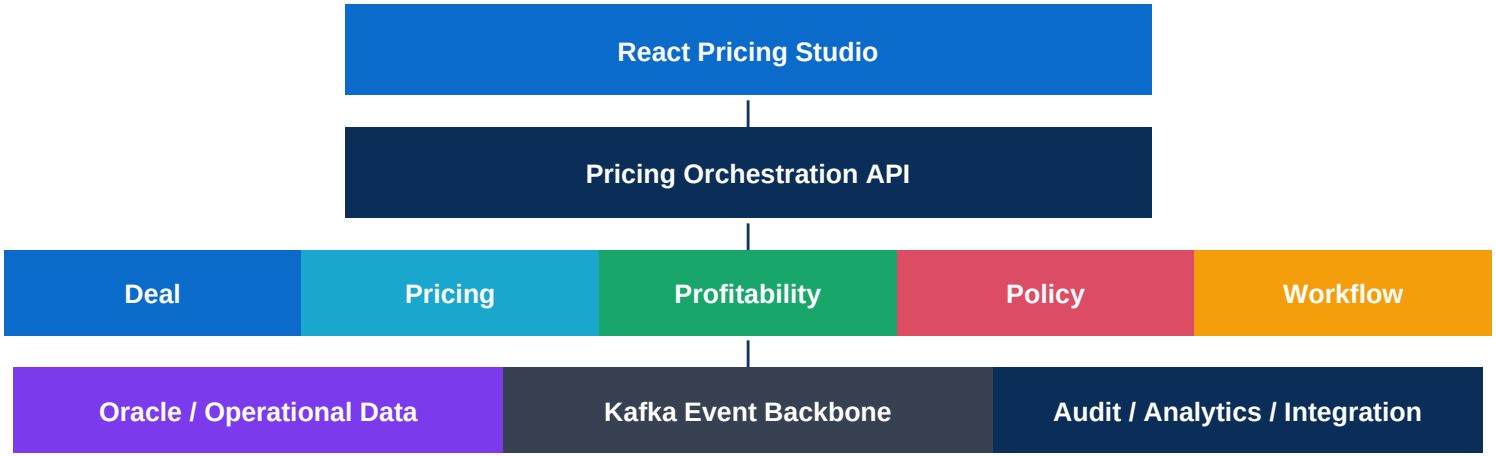
Interview line: “The UI is organized around what Sales is trying to accomplish - not around the names of backend services.”

3. Back-End Rationale: Separate the Business Decisions

Deal Service	Owns client/deal/product context and provides the Deal ID that ties the lifecycle together.
Pricing Engine	Determines what to charge using versioned fee models, tiers, discounts, bundles and overrides.
Scenario Engine	Evaluates multiple commercial alternatives without overwriting the base deal.
Profitability Engine	Calculates revenue, cost-to-serve, profit and margin for each scenario.
Policy Engine	Evaluates authority, thresholds, risk and regulatory guardrails; returns exceptions and required approvals.
Workflow Engine	Orchestrates Finance, Product and Risk approvals based on policy outcomes.
Audit Service	Creates traceability for pricing, scenario, policy and approval decisions.
Kafka	Publishes domain events so audit, analytics, notification and integrations remain decoupled.

The four questions to remember	**Pricing** What should we charge?	**Profitability** Do the economics work?	**Policy** Are we allowed to offer it?	**Workflow** Who can approve it?

4. Architecture Story to Use in the Interview



30-second rationale: I started with the user journey rather than the technology. Sales starts with a deal, configures fees, compares scenarios, understands profitability, checks policy and obtains approval. I mapped those decisions to clear backend capabilities. Pricing determines what we charge; Profitability determines whether the economics work; Policy determines whether the terms are allowed; Workflow determines who can approve exceptions. Kafka then decouples audit, analytics and downstream integrations. These are logical capability boundaries first - not a mandate to deploy every box as a microservice on day one.

Memorable line: “The frontend follows the user's journey; the backend follows the business decision boundaries.”